

The 4th International Conference on Electrical Engineering and Informatics (ICEEI 2013)

Source Code Editing Evaluator for Learning Programming

Timotius Nugroho Chandra*, Inggriani Liem

*School of Electrical Engineering and Informatics, Institut Teknologi Bandung
Bandung 40132, Indonesia*

Abstract

Each semester, School of Electrical Engineering and Informatics (SEEI) ITB manages ±400 students taking programming courses with many examinations and exercises but can only provides limited feedback and facilities. The number of PCs are much less than the students. Although most students have personal computing devices, those devices have various specifications that create difficulties in evaluation. To tackle these issues, we provide an integrated system for learning programming that covers source code editing, compiling, execution, submission and evaluation of the process and the result.

In this paper, we emphasize on source code editing environment named **Doppel** and **Ganger (D&G)**. **D&G** is a web based application aimed for monitoring coding process by recording typing activities. It also supports online compilation and execution using an autograder. We have experimented programming practice using this environment and it has shown that **D&G** can significantly support the learning process of programming.

© 2013 The Authors. Published by Elsevier Ltd. Open access under [CC BY-NC-ND license](#).

Selection and peer-review under responsibility of the Faculty of Information Science & Technology, Universiti Kebangsaan Malaysia.

Keywords: source code editor, programming course, programming best practice, integrated system, autograder, web application

1. Introduction

Programming practice and examination are mandatory for every computing students. Each year, SEEI ITB offers a programming course for 400 students in average [1]. The number of instructors and computer resources is quite

* Corresponding author. Tel.: +62-85-933107900; fax: +62-22-2534222.

E-mail address: timotius.nc@gmail.com

limited. Currently there are only 25-30 instructors, and programming practices/examinations are supported with only 110 computers.

The limited amount of computers and instructors causes two main problems: (1) Programming practices or examinations can not be handled in one shift at the same time. The instructors were forced to prepare different sets of problem and (2) students do not receive sufficient guidance.

Nowadays, majority of SEEI students have their own laptops and/or mobile devices. Those devices could be utilized to solve the problem of limited computer resources. Nevertheless, utilizing those devices creates another problem: the devices have different specifications that will affect the execution time and if students install different version of compilers, the source code could run on their own devices but it could fail during the grading process in the instructor's computer. We need a source code editor and compiler that provide a standardized environment.

In the early programming learning stage, it is important that the students get the guidance from the instructors in order to code well. Hence, the instructor's role is very much needed. The source code editor should also be able to do this role just as effective in order to reduce the workload of an instructor.

To provide a standardized environment and also guidance for the students, the source code editor shall (1) be independent to the platform (accessible via PC and/or mobile devices), (2) be able to grade source-code-typing process (to help the instructors), and (3) have a standardized compilation and execution. With this application, a programming practice or exam can take place everywhere. Students can use their laptops or mobile devices regardless of where they are and they can also get guidance in the form of grade from their typing-process.

2. Background Works

We explored four source code editors: touchqode [2], Codeanywhere [3], Ideone [4], and CodeMirror [5]. touchqode is an Android-based mobile application. touchqode has the capability to do syntax highlighting, code suggestion and File Transfer Protocol (FTP) access. touchqode provides two buttons to compile and execute over the internet and the result will be sent back to the sender. Programming languages supported by touchqode are HTML, PHP, JavaScript, Python, Ruby, Java, C#, and C/C++.

Codeanywhere is available in two versions: web and mobile. The user can store his source code locally or synchronize with its online version in the cloud. However, Codeanywhere does not provide the facility to run the source code. Programming languages supported by Codeanywhere are interpreted languages: PHP, HTML, CSS, JavaScript, and XML.

Ideone is a web-based source code editor. Ideone provides online compilation and execution features. There are currently 44 programming languages supported by Ideone.

CodeMirror is a JavaScript component in the form of source code editor. It can be attached to another web-based application. This component has the capability to do syntax highlighting for 26 programming languages. CodeMirror only acts as a source code editor and does not have the capability to do the compilation or execution. In our research, CodeMirror is used as the main component to do syntax highlighting to the source code written.

The summary of our exploration to the four source code editors are presented on Table 1.

Table 1 Capability Comparison from 4 Source Code Editors

Capability	touchqode	CodeAnywhere	Ideone	CodeMirror
Device supported	<i>Mobile</i>	PC and <i>web</i>	PC	PC
Compilation	Online	No	Online	No
Syntax Highlighting	Yes	Yes	Yes	Yes
Bracket matching	No	Yes	Yes	Yes
Open Source	No	No	No	Yes
Supported programming languages	HTML, PHP, JavaScript, Python, Ruby, Java, C#, C/C++	PHP, HTML, CSS, JavaScript, XML	Pascal, C/C++, Java, and 41 more.	Pascal, C/C++, Java, and 23 more.
Support to define new syntax highlighting	No	No	No	Yes

3. Architecture

D&G can be used from an independent computer or as part of an integrated system for learning programming. For SEEI ITB, D&G is designed to collaborate with an autograder in an environment named Oddysseus [6]. Students use Oddysseus to submit source code. Instructors use Oddysseus to grade students' submission. In a practical session, students start to type their source code using D&G, then submit it to Oddysseus, then compile and/or execute the source code. The results are sent via D&G to the students. After finishing the practice/exam, students are allowed to check their grades on Oddysseus.

Oddysseus provides online compilation and execution service. This service is limited to registered users. An application needs to be granted a privilege (in the form of a token) to access the service. D&G has the privilege to access the services (via HTTP protocol). D&G has two ways for compiling and executing source code: (1) using Oddysseus' services and (2) using their own machine (provided the machine has those capabilities). Therefore, students who do not have internet access can still write, compile, and execute their source code.

The architecture has three layers: source code editor, compilation/execution delegator, and compiler/executor layer. Source code editor layer handles the source code editing and the typing process grading. This source code editor layer is named Doppel. Compilation/execution delegator layer is responsible to delegate the compilation or execution to another system. This layer is named Ganger. Compiler/executor layer handles the compilation/execution.

D&G's architecture is presented on Fig. 1.

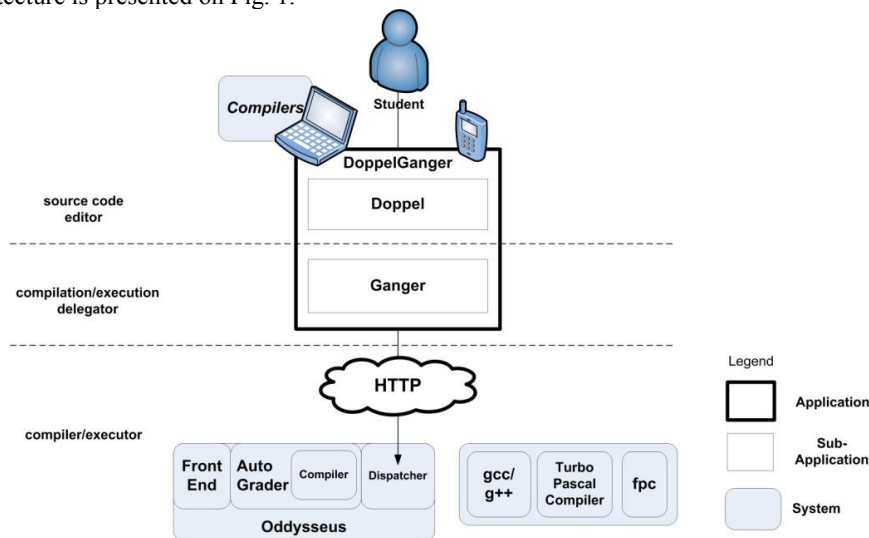


Fig.1. D&G Architecture

The compiler/executor can be a system which provides compilation/execution service or another computer installed with a compiler. In our experimentation, the compiler/executor layer will be handled by **Oddysseus** [6] to provide an integrated environment for SEEI ITB.

4. Architecture

D&G is not an editor for developer to build an application, it is designed for the purpose of learning how to program. With this specialty, D&G has its own requirements. There are two functional requirements for D&G: (1) source code editing and compilation/execution, (2) recording and grading the typing process. D&G has one non-functional requirement: deployable in PC and mobile device with minimum effort. These requirements are elaborated more in the following sections.

4.1. Source Code Editing and Compilation/Execution

Source code editing is the basic requirement for a source code editor. D&G provides source code editing functionality along with syntax highlighting. The syntax highlighting is based on the programming language of the source code. D&G will do the syntax highlighting for a predefined programming language. New programming language definition for syntax-highlighting can be added using an interface provided by CodeMirror [5]. Therefore, as a source code editor, Doppel supports every programming language as long as the syntax highlighting definition has been defined.

In the SEEI ITB environment, D&G delegates the compilation and execution to Odysseus. That way, D&G solves the problem of non-standard environment caused by various devices used by students because all submissions are graded using the same compiler. The compilation and execution services by Odysseus needs internet connection, which is always available during a programming class. Without internet connection, students can practice by themselves by compiling and executing with their own local compiler.

4.2. Recording and Grading The Typing Process

To monitor the source code typing process, Doppel records student's actions while the student is typing his source code by capturing the following data: keyboard press, mouse click, save, copy/paste, compile/execute, and the duration of typing. After the students finish typing the source code, Doppel evaluates the source code by examining these recorded action. From the recorded data, students obtain feedbacks directly from Doppel. With this capability, Doppel takes over a part of an instructor's role. Therefore, it contributes to solving the problem of limited instructors.

Doppel counts three keyboard press categories: (a) backspace-counter which is incremented by one every time students press backspace, (b) direction-counter which is incremented by one every time students press direction key (right/left/up/down), (c) button-counter which is incremented by one everytime a key except backspace is pressed.

Considering that Doppel is designed for learning programming, each student's action is recorded during Doppel usage and the actions are saved with the source code in a text file. When the text file is re-opened and the user rework on the source code, the recording process is continued from the last action contained within the file. All actions concerning a source code are always kept in a history. The text file produced from Doppel's recording is stored in JSON (JavaScript Object Notation) format [7]. As a consequence of this representation, students can not compile/executedirectly with a normal compiler. However, Doppel provides a feature for exporting the JSON formatted text file into a plain source code.

All Doppel evaluates the typing process in three aspects: efficiency, accuracy and duration. The first aspect is efficiency. Most of the work of a programmer when producing a code is typing. Efficiency is defined as the ratio between the number of characters in the source code and the total typing (key press) count. If the number of characters in the source code is much less than the total typing count it means that the programmer must have typed many useless characters or was just playing with the keyboard. Efficiency is calculated with Eq. (1).

$$Efficiency = \frac{\text{Characterscount in source code}}{\text{Total typing count}} \times 100\% \quad (1)$$

The second aspect is typing accuracy. If the number of characters in the source code is exactly the same as the number of key pressed, then the source code is typed in one shot. If the programmer types useless keys such as back space, mouse click or direction keys, then he does not type the program straightaway. The backward key is usually pressed for correcting a wrong word or statement. Accuracy calculated with Eq. (2).

$$Accuracy = \frac{\text{Total typing count} - \sum (\text{backspace} + \text{mouse click} + \text{directionkey})}{\text{Total typing count}} \times 100\% \quad (2)$$

The third aspect is the duration from the beginning of typing until the source code is saved. It reflects the speed of problem solving. This value must be used carefully. For simple problem, the duration must be short whilst for complex problem, a large value is still tolerable. This aspect is measured from the recording of typing duration and accumulated in the history of the source code. This aspect is calculated with Eq. (3).

$$\text{Speed of problemsolving and sourcecodetyping} = \text{Finishtime} - \text{Start time} \quad (3)$$

The three formulas explained in this section are simple, limited, and presented individually and calculated from raw recording data. The combination of values and final source code editing grade are not part of this research. However, a passing criteria and good coding practices can be determined from these three values, source code characteristics (small or big), complexity of the problem, and values set by instructor.

As example, for a small and simple program, a student will obtain a perfect score if the product (R1) and the editing process are good (R2).

R1. The rule for obtaining a good result/product is: if executions by autograder produce a correct result.

R2. The rules of good practices applied for good process are:

R2.1. if the value of typing count is almost the same as the character count, with a tolerance set by instructor.

R2.2. if the counter of save is small (a value set by instructor).

R2.3. if the counter of copy/paste is small (a value set by instructor).

R2.4. if the compilation/execution action is less than or equal 2 (or other value set by instructor).

R2.5. the working duration is short (set by instructor).

4.3. Deployment in PC and Mobile Devices with Minimum Effort

D&G must be deployable and accessible via PC or mobile devices. The first alternative is to develop two versions of application: a desktop version and a mobile version. The second alternative is to develop PC-web and mobile-web application. In this research, we chose the second alternative. This alternative has two advantages over the first one: (a) less number of application to be developed. (b) allows an easier access compared to using a desktop or mobile application.

The measurement of minimum effort for D&G development is based on the amount of reusable class: the less amount of class developed the less effort needed. In order to achieve this, we use Model View Controller (MVC) design pattern [8]. MVC design pattern is chosen because it enables the development to re-use as many class as possible to be deployed on web-PC and web-mobile all at once. After the development, 89% of classes are reused for both web-PC and web-mobile version.

5. Implementation

D&G is implemented using JavaScript (version 1.8) and HTML (version 5.0). The hardware environments are: Intel Core i3-2330M @2.20 GHz processor, 4096 MB memory, and 530 GB harddisk. The software environments are: Windows 7 64-bit operating system, jQuery library (version 1.7.2), AngularJS framework (version 1.7.2), NetBeans IDE (version 6.9.1), and Google Chrome web browser (version 21).

The platform for D&G's development is depicted on Fig. 2. On Doppel, we use CodeMirror component to do syntax-highlighting. CodeMirror is chosen because it is open source and we can configure it according to our needs.



Fig.2. Application Platform

AngularJS framework is chosen because it applies the MVC design pattern [9] which is suitable for web application development. Moreover, AngularJS also utilizes jQuery library which can speed up the development process.

For implementing the compilation and execution capabilities, Ganger uses Oddysseus' services by sending messages through HTTP protocol. The messages sent are adjusted to conform the format specified by Oddysseus. According to the Oddysseus' software design, messages sent to Oddysseus from Ganger is equipped with a specific token. The token is used to prevent service misuse or access from unknown application. Only particular application with a valid token can access Oddysseus' services.

6. Testing

Functional testing has been done in several phases, from unit testing to usability testing in real environment. Firstly unit testing is performed using JsTestDriver [10]. Each class has been unit-tested before it is integrated to D&G and the integrated environment (with Oddysseus [6]). After unit test, functional test on PC and mobile devices are executed for editing, recording the data, and collaboration with the integrated environment. User can edit, get feedback based on the recorded data, and submit to the integrated environment as well as to compile and execute in the local device with available local compiler.

D&G has also been tested for real user (students) during the first semester of academic year 2012/2013, for a small class in Algorithm and Programming course with 25 participants. Although this class is only a subset of the target students, the experiment showed that D&G can be used on a real programming course for supporting programming practice and examination. Participants successfully created source code in Pascal, C, and C++, took benefit from the syntax highlighting and the grading of typing process, saved and re-opened/edited the source code, and also compiled/executed it using Oddysseus' services. The utilization by big number of students (± 400 as mentioned in the Introduction) will not influence D&G performance and functionalities since it is installed individually in each student device. The big number of user will only influence the submission and grading process that will be handled by the scalability of Oddysseus [6].

We tested the non-functional requirement by testing the deployability on PC and mobile devices. The test gave data of reusable class that will reflect programming effort. It has been shown that 89% of the class is used by both platforms. The effort of porting from one platform to another is to change the user interface (viewer) class and suppress the capability that is not implementable yet, such as syntax highlighting in mobile devices.

7. Evaluation and Discussion

The main contribution of this research is creating a software design and implementation of a source code editor and evaluator which can be accessed via browser, from PC and mobile devices. D&G and its integrated environment shall be able to solve the problem of big class in a programming course. In this current version, the capability of

source code editing is not much different than the other source code editor. In the future, we plan to implement a source code editor that shall be able to integrate small portion of source code sent by a very simple device such as mobile phone via short message service (SMS). The system will then compose that small portions of code into a complete program. This allows our students to code anywhere and anytime because sending SMS via mobile phone does not need an internet connection. It will be useful when internet connection is a constraint.

In learning programming, coding process is as important as the result (the source code) itself. Without tools, the instructor must continually stay next to the students to help oversee the coding process. In our environment, **D&G** can substitute a part of instructors role and the students can do more practice individually. The limited number of human assistant can then focus more on students with bigger problem in learning programming since **D&G** captures the typing process that will be used as a monitoring assistant. Furthermore, this research contributes to the analysis of programming best practice for typing-process grading. This analysis can still be elaborated more on another research by processing the data of user's action. Evaluation of coding process explained in section 4.1 is simplified and limited on keypresses. A further study is still needed for analyzing combination cases from recorded data and it can be complicated. For example, a few save actions do not always guarantee a good programming practice. However for large-sized source code, many save actions actually indicate a good incremental programming practice. Therefore, the grading formulation needs to be further elaborated by considering more aspects. Other means to capture user behavior, to record more keypress, and to relate to predefined objective can still be developed to improve the grading as part of the automated evaluation of programming process.

Acknowledgements

We would like to thank Karol Danutama for providing an opportunity to collaborate via *Odyssey*.

References

- [1] ITB, D.P., Daftar Peserta Kelas (DPK) SEMESTER 1, TAHUN 2011/2012. Retrieved August 24 2012, from Akademik ITB: https://six.akademik.itb.ac.id/publik/daftarkelas.php?ps=tpb&semester=1&tahun=2011&th_kur=2008. 2012.
- [2] Touchqode.. touchqode – mobile code editor. Retrieved January 3 2012, from touchqode – mobile code editor: <http://www.touchqode.com>. 2012.
- [3] Ademptio. Online Code Editor – CodeAnywhere. Retrieved January 3 2012, from Online Code Editor – CodeAnywhere: <http://codeanywhere.net>. 2012.
- [4] Labs, S.R., Ideone.com | Online IDE & Debugging Tool. Retrieved January 3 2012, from Ideone.com | Online IDE & Debugging Tool: <http://ideone.com>. 2012.
- [5] Haverbeke, M. CodeMirror. Retrieved August 7 2012, from CodeMirror: <http://codemirror.net>. 2012.
- [6] Danutama, K. Pengembangan Autograder Kompetisi Untuk Penyelenggaraan Kuliah Pemrograman”. Thesis: Institut Teknologi Bandung, Bandung. 2012.”
- [7] Crockford, D. RFC 4627 – The application/json Media Type for JavaScript Object Notation (JSON). Retrieved February 22 2013, from RFC 4627: <http://tools.ietf.org/html/rfc4627>. 2006.
- [8] Gamma, Erich., Helm, Richard., et.al. “Design Patterns: Elements of Reusable Object-Oriented Software”. Addison-Wesley, USA. p. 14-16. 1995.
- [9] AngularJS: Developer Guide. Retrieved August 15 2012, from AngularJS: <http://docs.angularjs.org/guide>. 2012.
- [10] Smith, C. js-test-driver-Remote JavaScript Console. Retrieved September 1 2012, from js-test-driver: <http://code.google.com/p/js-test-driver>. 2012.